

06 union find

Union-Find (Disjoint Set Union / DSU)

1. What is it, and when is it used?

Union-Find (Disjoint Set Union, DSU) is a data structure that maintains a collection of disjoint (non-overlapping) sets and supports two near- $O(1)$ operations: `find(x)` (which set does x belong to?) and `union(x, y)` (merge the sets containing x and y). It's used whenever a problem is fundamentally about **dynamic connectivity / grouping** — “are these two things in the same group?” — especially when the graph/edges arrive incrementally.

2. Why is it used?

Re-running BFS/DFS from scratch every time you add an edge and ask “are these connected?” costs $O(V+E)$ per query. Union-Find, with path compression and union by rank/size, answers both `find` and `union` in **amortized nearly $O(1)$** (technically $O(\alpha(n))$, the inverse Ackermann function — effectively constant for all realistic input sizes).

3. Where is it used?

- Connected components in an undirected graph
- Cycle detection in undirected graphs
- Kruskal's Minimum Spanning Tree algorithm
- Number of islands (grid connectivity, alternative to BFS/DFS)
- Accounts merge / friend circles / redundant connection
- Network connectivity queries (dynamic, edges added over time)
- Percolation problems
- Detecting if a graph becomes fully connected after adding edges one at a time

4. How do you identify it's a Union-Find problem?

Signal phrases (2-minute test): - “Are X and Y connected?” repeated across many queries - “Number of connected components” (especially with edges added incrementally) - “Redundant connection” / “find the edge that creates a cycle” - “Merge accounts/groups that share a common element” - “Minimum spanning tree” (Kruskal's specifically) - “Group friends/islands/provinces” by transitive relationships - Edges/relationships given as a list of pairs, and you need grouping, not shortest paths

5. Can you identify it from constraints?

Constraint clue	Implication
Edges/unions arrive incrementally (process one at a time, need answer after each)	Union-Find shines — online connectivity
Just need “connected or not” / “how many groups”, not path length or weights	Union-Find, not Dijkstra/BFS
n up to 10^5 - 10^6 elements, many union/find ops	Need path compression + union by rank for near- $O(1)$
Graph is undirected	Union-Find is a natural fit (doesn't handle directed reachability well)
Asked for MST	Kruskal's = sort edges by weight + Union-Find to avoid cycles

6. Types / Variants

1. **Quick Find** — `find` is $O(1)$ but `union` is $O(n)$ (naive, rarely used in interviews).
2. **Quick Union** — `union` links tree roots; `find` can degrade to $O(n)$ without balancing.
3. **Union by Rank/Size** — always attach the smaller tree under the bigger one's root, keeping trees shallow ($O(\log n)$ worst case).
4. **Path Compression** — during `find`, make every node on the path point directly to the root, flattening future lookups.
5. **Union by Rank + Path Compression (Weighted Union-Find)** — the gold standard, gives amortized $O(\alpha(n)) \approx O(1)$.
6. **Weighted Union-Find with relationship values** — stores a relative “weight” between element and its parent (used in problems like “evaluate division” / ratio queries).

7. Rationale / Intuition

Think of each set as a tree, where the root is the set's “representative.” `find(x)` walks up parent pointers until it hits a root. Two elements are in the same set iff they share the same root. Naively this can degrade into a long chain ($O(n)$ per find), so two optimizations fix it: - **Union by rank/size**: always attach the shorter tree under the taller one's root, so tree height grows only logarithmically. - **Path compression**: while doing `find`, re-point every visited node directly to the root, so future queries on those nodes are $O(1)$.

Together, these two tricks give amortized almost-constant time per operation — one of the most elegant data structure results in CS.

8. Time & Space Complexity

Operation	Time (with both optimizations)	Space
<code>find(x)</code>	$O(\alpha(n)) \approx O(1)$ amortized	$O(n)$
<code>union(x, y)</code>	$O(\alpha(n)) \approx O(1)$ amortized	$O(n)$
Build (n elements)	$O(n)$	$O(n)$
Kruskal's MST (using DSU)	$O(E \log E)$ (dominated by edge sort)	$O(V)$

Without optimizations: `find` / `union` can degrade to $O(n)$ worst case (a long chain).

9. Comparison with Other Patterns

Pattern	Answers	Handles incremental edges?	Handles weighted shortest path?
Union-Find	“Same group?” / count of groups	Yes, efficiently (online)	No
BFS/DFS	Connectivity, shortest path (unweighted), traversal order	Recompute from scratch each time (offline)	BFS handles unweighted shortest path only
Dijkstra	Shortest path, weighted, non-negative	N/A	Yes
Kruskal's (uses Union-Find)	Minimum Spanning Tree	N/A	Uses edge weights for MST, not shortest path
Tree Traversal	Structure of a single tree	N/A	N/A

Rule of thumb: if edges/relationships are being added dynamically and you repeatedly need “are these connected / how many groups,” Union-Find beats re-running BFS/DFS every time.

10. Reusable Java Templates

10.1 Union-Find with Path Compression + Union by Rank

```
class UnionFind {
```

```

private final int[] parent;
private final int[] rank;
private int components;

public UnionFind(int n) {
    parent = new int[n];
    rank = new int[n];
    components = n;
    for (int i = 0; i < n; i++) parent[i] = i;
}

public int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]); // path compression
    }
    return parent[x];
}

public boolean union(int x, int y) {
    int rootX = find(x), rootY = find(y);
    if (rootX == rootY) return false; // already connected -> would form a cycle

    if (rank[rootX] < rank[rootY]) {
        parent[rootX] = rootY;
    } else if (rank[rootX] > rank[rootY]) {
        parent[rootY] = rootX;
    } else {
        parent[rootY] = rootX;
        rank[rootX]++;
    }
    components--;
    return true;
}

public boolean connected(int x, int y) {
    return find(x) == find(y);
}

public int countComponents() {
    return components;
}
}

```

10.2 Union-Find by Size (alternative to rank, also gives set sizes)

```

class UnionFindBySize {
    private final int[] parent;
    private final int[] size;

    public UnionFindBySize(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }

    public int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]]; // path halving
            x = parent[x];
        }
        return x;
    }

    public void union(int x, int y) {

```

```

        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return;
        if (size[rootX] < size[rootY]) { int t = rootX; rootX = rootY; rootY = t; }
        parent[rootY] = rootX;
        size[rootX] += size[rootY];
    }

    public int sizeOf(int x) { return size[find(x)]; }
}

```

10.3 Kruskal's MST using Union-Find

```

import java.util.*;

class KruskalMST {
    public int minimumSpanningTree(int n, int[][] edges) {
        // edges[i] = {u, v, weight}
        Arrays.sort(edges, (a, b) -> a[2] - b[2]);
        UnionFind uf = new UnionFind(n);
        int totalWeight = 0, edgesUsed = 0;

        for (int[] edge : edges) {
            if (uf.union(edge[0], edge[1])) {
                totalWeight += edge[2];
                edgesUsed++;
                if (edgesUsed == n - 1) break;
            }
        }
        return edgesUsed == n - 1 ? totalWeight : -1; // -1 if graph not fully connectable
    }
}

```

11. Quick Recognition Checklist (2-minute test)

- ☐ Problem talks about “groups”, “connected components”, or “merging sets”?
- ☐ Edges/relations arrive one at a time and you need answers along the way?
- ☐ Asked for MST, or “redundant edge that creates a cycle”?
- ☐ No need for path length/weight — just group membership?

If 2+ checked → Union-Find.